How `generator.py` and `api.py` Work — Full Walkthrough

The big picture first

Before any code, here's the actual journey a single question takes, end to end:

```
React (or Postman/curl for now)
   │  HTTP POST {"question": "..."}
   ▼
Express — queryController.js (askQuery function)
   │  axios.post("http://localhost:8000/rag/query", {question})
   ▼
FastAPI — api.py  (THIS is what we're walking through)
   │  1. Validate the incoming JSON
   │  2. Turn the question into a vector, search FAISS for similar chunks
   │  3. Hand those chunks + the question to generator.py
   ▼
generator.py
   │  Feed question + chunks into TinyLlama, get back generated text
   ▲
   │  return {"answer": "..."}
Express receives it, saves to Postgres, sends to React
```

Two completely separate Python files do two completely separate jobs:

- `api.py` — the *traffic controller*. It receives requests, calls the right functions in the right order, and sends back a response. It does not do any AI work itself.
- `generator.py` — the *actual AI*. It takes text in, runs it through a language model, and produces new text out. It has zero idea that HTTP or Express even exist — it's just a Python class anyone could call.

This separation matters: you could swap TinyLlama for a different model by only touching `generator.py`, and `api.py` would never need to change.

File 1: `generator.py`

The imports

```python
from typing import List
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
```

- `List` — purely for documentation. When you see `context_chunks: List[str]` later, it's telling *you* (and your editor) "this argument should be a list of strings." Python doesn't actually enforce this at runtime — it's a hint, not a rule.
- `torch` — PyTorch, the underlying engine that actually does the math (matrix multiplications) that makes a neural network run.
- `AutoTokenizer`, `AutoModelForCausalLM` — both from HuggingFace's `transformers` library. "Auto" means: you give it a model's name, and it figures out the right tokenizer/model class automatically, instead of you having to know TinyLlama's specific architecture by hand.

`class Generator:` and `__init__`

```python
class Generator:
    def __init__(self, model_name: str = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"):
```

This is the constructor — code that runs once, the moment someone writes `Generator()`. `model_name` has a default value, so calling `Generator()` with no arguments automatically uses TinyLlama. If your team later trains a custom model, you'd call `Generator(model_name="your-custom-model-path")` instead — nothing else changes.

```python
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
```

A **tokenizer** converts human text into numbers (and back). Neural networks can't read words — they read numbers. `from_pretrained(model_name)` downloads TinyLlama's specific tokenizer (every model has its own — they split words differently) the first time, then caches it locally after that.

```python
        self.model = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype=torc
```

This downloads and loads the actual model — all 1.1 billion learned numbers (parameters) that make up TinyLlama. `torch_dtype=torch.float32` controls the *precision* each number is stored at — float32 is more precise but uses twice the memory of `bfloat16`. This is the line we just changed to fix the crash.

"CausalLM" means "Causal Language Model" — a model that predicts the *next* word based only on the words that came *before* it (not after). That's how all chat models like this work: predict one next word, add it to the text, predict the next one, repeat.

```python
```

```
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model.to(self.device)
```

`cuda` means "NVIDIA GPU." This checks if you have a GPU PyTorch can use — if not, it falls back to your regular CPU. `.to(self.device)` actually moves all the model's numbers into that hardware's memory so it can compute there. On your machine, this is almost certainly just `"cpu"`, which is fine — just slower than a GPU would be.

`generate_answer` — the actual work function

```python
    def generate_answer(self, query: str, context_chunks: List[str], max_new_tokens: i
```

This is the only method anyone outside this class actually calls. `query` is the user's question. `context_chunks` is the list of relevant text snippets that retrieval already found. `max_new_tokens` caps how long the generated answer can be (200 tokens ≈ roughly 150 words) — without a cap, the model could ramble indefinitely.

```python
        context = "\n\n".join(context_chunks) if context_chunks else "No relevant cont
```

`"\n\n".join([...])` glues a list of strings together into one string, with a blank line between each — turning 5 separate retrieved chunks into one combined block of text. The `if/else` handles the edge case where retrieval found nothing at all.

```python
        messages = [
            {"role": "system", "content": "..."},
            {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"},
        ]
```

This is the standard format chat models expect: a list of role/content pairs. `"system"` sets the model's overall behavior/personality (here: "only use the given context, don't make things up" — this is what keeps it grounded instead of hallucinating). `"user"` is the actual question, with the retrieved context stitched in right before it, so the model has everything it needs in one place. The `f"..."` is an f-string — it inserts the real values of `context` and `query` directly into that text.

```python
        prompt = self.tokenizer.apply_chat_template(messages, tokenize=False, add_gene
```

Different models were trained on different exact text formats for conversations (special marker tokens, specific ordering, etc.). `apply_chat_template` knows TinyLlama's specific format and converts your clean `messages` list into the exact raw string TinyLlama expects — without you having to know or hardcode that format yourself. `add_generation_prompt=True` adds the final bit that tells the model "now it's your turn to respond."

```python
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.device)
```

This actually runs the tokenizer — converting that formatted text string into numbers (tensors). `return_tensors="pt"` means "give me PyTorch tensors" (as opposed to some other library's format). `.to(self.device)` moves those numbers onto the same hardware (CPU/GPU) the model lives on — they have to match, or PyTorch will error.

```python
        with torch.no_grad():
            output_ids = self.model.generate(...)
```

`torch.no_grad()` tells PyTorch "don't bother tracking how to compute gradients for this." Gradients are only needed during *training* (to learn from mistakes) — during inference (just generating an answer), skipping this tracking makes things faster and uses less memory, for zero downside.

`model.generate(...)` is the actual generation step — repeatedly predicting the next most likely token, adding it, and predicting again, until it decides it's done or hits `max_new_tokens`. `do_sample=False` means "always pick the single most likely next word" (deterministic, reproducible) rather than randomly sampling among likely options (which would make answers vary each time you ask the same question).

```python
        new_tokens = output_ids[0][inputs["input_ids"].shape[1]:]
        return self.tokenizer.decode(new_tokens, skip_special_tokens=True).strip()
```

`model.generate()` actually returns *your entire prompt plus the new answer* stuck together as one sequence of numbers. This line slices off only the part that came *after* your original input — i.e., just the newly generated answer. `decode(...)` converts those numbers back into human-readable text. `skip_special_tokens=True` removes formatting markers that aren't meant to be shown to a user. `.strip()` just trims stray whitespace.

File 2: `api.py`

Imports

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

`FastAPI` — the class you instantiate to create your web server. `HTTPException` — lets you deliberately return an error response (like "400 Bad Request") with a specific message, instead of crashing. `BaseModel` — from `pydantic`, FastAPI's data-validation library; you inherit from it to define exactly what shape an incoming request must have.

```python
from src.ingestion.loader import DocumentLoader
from src.chunking.chunking import RecursiveChunker
from src.embedding.embedding import Embedding
from src.vectorstore.vectore_store import VectorStore
from src.retrieval.vector.faiss_retriever import SearchFaiss
from generator import Generator
```

These pull in your team's original RAG modules plus your new `Generator` class. **Honest flag:** `DocumentLoader` and `RecursiveChunker` are imported here but never actually used anywhere in this file right now — they're leftover from an earlier version where the API also handled document uploads. Right now, ingestion only happens via `main.py`, separately. They're harmless to leave (just unused), but worth knowing they're not doing anything currently.

The module-level setup

```python
app = FastAPI()

embedder = Embedding()
vectordb = VectorStore()
retriever = SearchFaiss(embedder)
```

This code runs **exactly once** — the moment the file is first loaded when you run `uvicorn api:app`. This is deliberate and important: `Embedding()` loads a model into memory; if this lived *inside* the route function instead, it would reload that model on every single request, making each call take far longer than necessary. Creating it once, here, means every request reuses the same already-loaded objects.

```python
```

```
try:
    vectordb.load()
    print(f"Loaded index with {vectordb.index.ntotal} vectors")
except Exception:
    print("No index found yet — run main.py first to build one.")
```

`try/except` means "attempt this, and if it fails, don't crash — do something else instead." `vectordb.load()` reads the FAISS index file `main.py` already saved to disk. If that file doesn't exist yet (you haven't run `main.py`), this would normally throw an error and stop everything — the `except` catches that and just prints a warning instead, so your server still starts (it just won't be able to answer questions yet).

```
python

generator = None

def get_generator():
    global generator
    if generator is None:
        generator = Generator()
    return generator
```

This is a pattern called **lazy loading**. `generator` starts as `None` — TinyLlama is *not* loaded when the server starts. The function `get_generator()` checks: "has anyone created the real Generator yet?" If not, create it now (this is the slow part — downloading/loading the model); if it already exists, just hand back the existing one. `global generator` is required because we're *reassigning* a variable that lives outside this function — without it, Python would think you're creating a brand new local variable instead of updating the outer one.

Why bother with this instead of just creating `Generator()` up top with everything else? Because loading TinyLlama is slow (and the first time, requires downloading ~2.2GB). If we did it eagerly, *starting your server* would hang for minutes every single time, even if nobody's asked a question yet. Lazy loading means: pay that cost once, only when actually needed.

The request shape

```
python

class ChatRequest(BaseModel):
    question: str
```

This says: "a valid request to `/rag/query` must contain a field called `question`, and it must be a string." If someone sends `{"question": 123}` (a number, not a string) or omits the field entirely, FastAPI automatically rejects it with a clear error — before your function even runs. You don't write any of that checking yourself.

The routes

```python
@app.get("/")
def root():
    return {"status": "API is running"}
```

This is what answered when you hit the bare root URL earlier — purely a "is this thing alive" check, unrelated to the actual RAG logic.

```python
@app.post("/rag/query")
def rag_query(req: ChatRequest):
```

`@app.post("/rag/query")` registers this exact URL+method combination. This needed to match `/rag/query` exactly because that's the literal URL your existing `queryController.js` already calls via `axios.post`. `req: ChatRequest` means FastAPI automatically parses the incoming JSON body and hands you back a `req` object where `req.question` just works as a normal Python attribute — no manual `json.loads()` or key-checking needed.

```python
    if vectordb.index is None or vectordb.index.ntotal == 0:
        raise HTTPException(status_code=400, detail="No documents indexed yet — run ma
```

A safety check — if nothing's been indexed (either `vectordb.load()` failed earlier, or it loaded an empty index), don't try to search an empty/nonexistent index and crash confusingly. Instead, return a clear, deliberate error.

```python
    results = retriever.search_context(req.question, vectordb.index, vectordb.metadata
    context_chunks = [r["text"] for r in results]
```

This calls your team's original retrieval function — embed the question, find the 5 closest matching chunks in the FAISS index. `results` comes back as a list of dictionaries (each with `text`, `source`, `page`, `score` — you saw this exact shape in your `main.py` test output). The second line is a **list comprehension** — a compact way of writing "build a new list by taking `r["text"]` out of each `r` in `results`." It's equivalent to:

```python
```

```
context_chunks = []
for r in results:
    context_chunks.append(r["text"])
```

just shorter.

```python
    gen = get_generator()
    answer = gen.generate_answer(req.question, context_chunks)

    return {"answer": answer}
```

First call ever to `get_generator()` is the moment TinyLlama actually loads (the slow part you experienced). `generate_answer(...)` is the method from `generator.py` we just walked through — feed it the question and the retrieved text, get back generated text. The final `return {"answer": answer}` is a plain Python dict — FastAPI automatically converts it to JSON for the response, matching exactly the `{"answer": "..."}` shape `queryController.js` already expects from the mock server.

How this actually connects to your Express backend — concretely

Your `queryController.js` already has this:

```js
const response = await axios.post("http://localhost:8000/rag/query", {
  question: question
});
const dummyResponse = response.data.answer;
```

Walk through what happens now that real `api.py` is running on port 8000 instead of the mock `index.js`:

1. `axios.post(url, {question: question})` sends an HTTP POST with JSON body `{"question": "..."}` to `localhost:8000/rag/query`
2. Your FastAPI `rag_query(req: ChatRequest)` function receives exactly that JSON, validated automatically
3. It runs retrieval + generation as walked through above
4. It returns `{"answer": "..."}` as JSON
5. Back in Node, `response.data.answer` pulls the real generated string out of that response — `response.data` is the parsed JSON body, `.answer` grabs that one field
6. Node saves it to Postgres (`chat_history` table) and sends it onward to whatever called the Express route

The reason zero changes were needed in `queryController.js` or `index.js`: Node doesn't know or care that Python is on the other end. `axios.post` just sends bytes over a network connection to a URL — it would behave identically whether `localhost:8000` was running Python, Node, Go, or anything else, as long as the URL, the request shape, and the response shape all match. That's the entire point of building services that talk over HTTP instead of directly importing each other's code.